

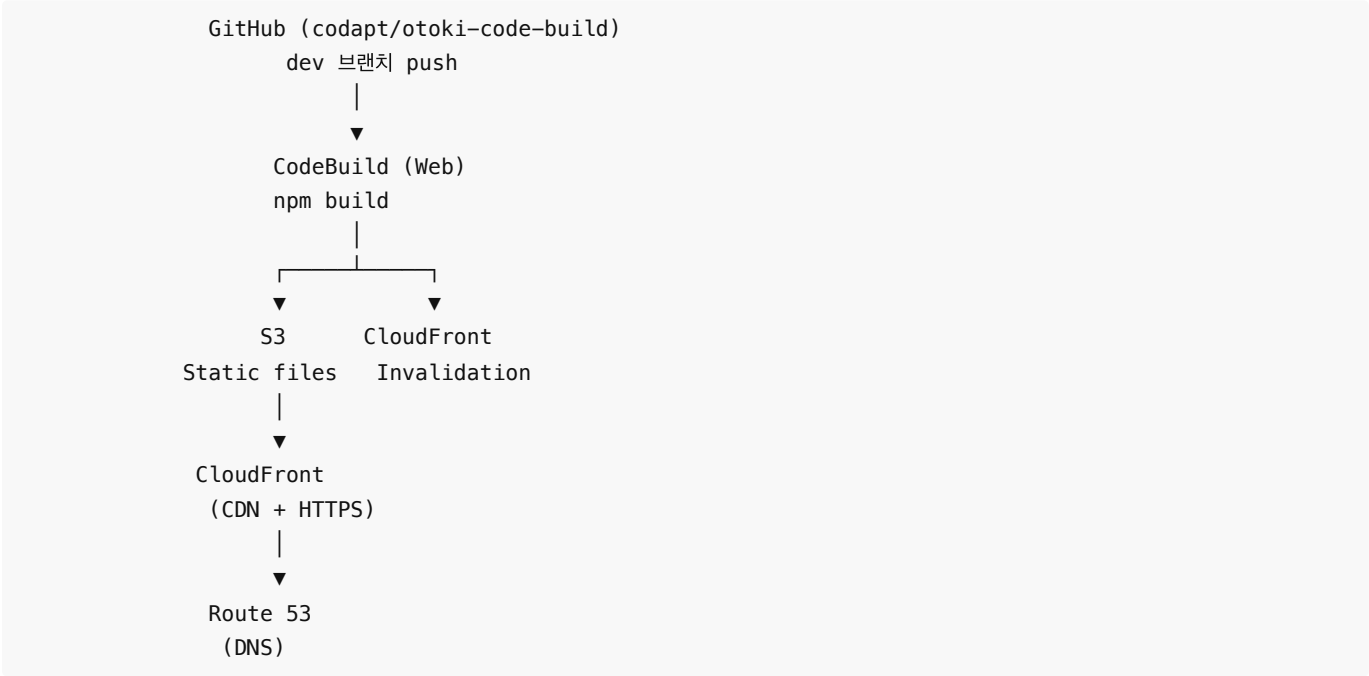
otoki-code-build AWS 인프라 가이드 — Web (Frontend)

0. 전체 아키텍처

프로젝트 개요

항목	내용
저장소	GitHub codapt/otoki-code-build (모노레포)
Frontend	React 19 + TypeScript 6 + Vite 8 (CSR)
리전	ap-northeast-2 (서울)
Stage	dev, prod

아키텍처 다이어그램



전제 조건

Backend 인프라(infra-backend.md)가 먼저 완료되어야 한다. VPC, Route 53 호스팅 영역 등 공통 리소스가 이미 존재해야 한다.

항목별 작업자 배정

섹션	항목	작업자	비고
2	S3 버킷 생성	인프라	정적 파일 호스팅용
3	CloudFront 배포 생성	인프라	S3 앞단 CDN (OAC 자동 생성)
4	S3 버킷 정책 설정	인프라	CloudFront Distribution ID 필요
5.1	IAM (Web CodeBuild Role)	인프라	S3, CloudFront Distribution ID 필요
5.2	IAM (Web Pipeline Role)	인프라	CodeBuild, Pipeline Artifacts, SNS 필요
6	ACM 인증서 (us-east-1)	인프라	CloudFront용 SSL 인증서

7	Route 53 + CloudFront 도메인 연결	인프라	ACM 인증서, CloudFront 도메인 필요
8	CI/CD (Web CodePipeline + CodeBuild)	앱	IAM Role, S3, CloudFront 필요
9	CloudWatch	앱	모든 리소스 생성 후
10	Dev vs Prod 요약	공통	참고용

앱 작업자 IAM 권한

AWS 관리형 정책 (3개):

정책	용도
AWSCodeBuildAdminAccess	CodeBuild 프로젝트 관리 (섹션 8)
AWSCodePipeline_FullAccess	CodePipeline 관리 (섹션 8)
CloudWatchFullAccess	CloudWatch 알람 + 로그 관리 (섹션 9)

인라인 정책 (1개, 이름: **dev-otk-pwrs-web-app-developer-policy**):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PassRole",
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam::<ACCOUNT_ID>:role/*-otk-pwrs-web-build-role"
    },
    {
      "Sid": "CodeConnections",
      "Effect": "Allow",
      "Action": "codeconnections:*",
      "Resource": "*"
    }
  ]
}
```

PassRole : CodeBuild 프로젝트 생성 시 서비스 역할을 연결하기 위해 필요. **CodeConnections** : CodeBuild 소스에 GitHub Connection 을 지정하기 위해 필요.

셋업 순서

인프라 작업자와 앱 작업자가 분리되어 있다. 의존성에 따라 아래 순서대로 진행한다.

[인프라 작업자]

2. S3 버킷 생성 ← 정적 파일 호스팅용
3. CloudFront 배포 생성 ← S3 앞단 CDN (OAC 자동 생성) → DISTRIBUTION_ID 확보
4. S3 버킷 정책 설정 ← DISTRIBUTION_ID 필요
5. IAM (Web CodeBuild + Pipeline Role) ← DISTRIBUTION_ID 필요
6. ACM 인증서 (us-east-1) ← CloudFront용 SSL 인증서
7. Route 53 (DNS) ← ACM 인증서, CloudFront 도메인 필요

[앱 작업자]

8. CI/CD (Web CodePipeline + CodeBuild) ← IAM Role, S3, CloudFront 필요
9. CloudWatch ← 모든 리소스 생성 후

1. 네이밍 컨벤션 & 태깅

패턴

{stage}-otk-pwrs-{service}

예: dev-otk-pwrs-web , prod-otk-pwrs-web-build

리소스 이름 전체 목록

리소스	Dev	Prod
S3 (Frontend)	dev-otk-pwrs-web	prod-otk-pwrs-web
CloudFront	dev-otk-pwrs-web-cf	prod-otk-pwrs-web-cf
CloudFront OAC	(자동 생성)	(자동 생성)
CodeBuild (Web)	dev-otk-pwrs-web-build	prod-otk-pwrs-web-build
CodePipeline (Web)	dev-otk-pwrs-web-pipeline	prod-otk-pwrs-web-pipeline
IAM - CodeBuild Role (Web)	dev-otk-pwrs-web-build-role	prod-otk-pwrs-web-build-role
IAM - CodePipeline Role (Web)	dev-otk-pwrs-web-pipeline-role	prod-otk-pwrs-web-pipeline-role
Route 53 Record	dev-powersales.otoki.com	powersales.otoki.com

태그 정책

모든 리소스에 아래 태그를 부여한다.

Key	Dev	Prod
Stage	dev	prod
Project	otk-pwrs	otk-pwrs
ManagedBy	terraform	terraform

2. S3 버킷 (Frontend 정적 파일)

2.1 S3 버킷 생성

콘솔: S3 > Create bucket

설정	Dev	Prod
Bucket name	dev-otk-pwrs-web	prod-otk-pwrs-web
Region	ap-northeast-2	ap-northeast-2
Object Ownership	ACLs disabled (BucketOwnerEnforced)	ACLs disabled (BucketOwnerEnforced)
Block public access	모두 ON	모두 ON

Versioning	Disabled	Disabled
Encryption	SSE-S3	SSE-S3

CloudFront OAC를 통해서만 접근하므로 퍼블릭 액세스를 완전히 차단한다. Frontend 정적 파일은 버전 관리가 불필요하다. 빌드마다 전체 교체(`s3 sync --delete`)한다.

3. CloudFront (CDN)

3.1 CloudFront 배포 생성

콘솔: CloudFront > Distributions > Create distribution

Origin 설정

설정	Dev	Prod
Origin domain	dev-otk-pwrs-web.s3.ap-northeast-2.amazonaws.com	prod-otk-pwrs-web.s3.ap-northeast-2.amazonaws.com
Origin access	Grant CloudFront access to origin: Yes	Grant CloudFront access to origin: Yes

CloudFront 배포 생성 시 "Grant CloudFront access to origin"을 Yes로 선택하면 OAC가 자동 생성된다. CloudFront 생성 후 "Copy policy" 버튼으로 S3 버킷 정책을 복사하여 4장의 버킷 정책에 적용한다.

Default cache behavior

설정	Dev	Prod
Viewer protocol policy	Redirect HTTP to HTTPS	Redirect HTTP to HTTPS
Allowed HTTP methods	GET, HEAD	GET, HEAD
Cache policy	CachingOptimized	CachingOptimized
Compress objects	Yes	Yes

추가 오리진 & Behavior — `/api/*` → ALB

브라우저 CORS 발생을 원천 차단하기 위해 CloudFront 에 두 번째 오리진(ALB) 을 등록하고 `/api/*` 경로를 프록시한다. 프론트는 상대 경로 (`/api/...`) 로 호출 → same-origin.

추가 Origin

설정	값
Origin domain	{api_fqdn} (예: dev-powersalesapi.codapt.kr)
Protocol	HTTPS only, TLSv1.2
비고	Route53 alias → EB ALB. ACM 인증서 SNI 일치

Ordered cache behavior (우선순위 상위)

설정	값
Path pattern	<code>/api/*</code>
Origin	ALB custom origin
Viewer protocol policy	Redirect HTTP to HTTPS

Allowed HTTP methods	GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE
Cache policy	CachingDisabled (관리형)
Origin request policy	AllViewer (Authorization/Cookie/Query 전달)
Compress objects	Yes

SAP 등 외부 서버-to-서버 소비자는 CloudFront 를 거치지 않고 `{api_fqdn}` 을 직접 호출한다 (CORS 무관). 웹 브라우저만 CloudFront 경유로 same-origin 혜택을 받는다.

Settings

설정	Dev	Prod
Distribution name	dev-otk-pwrs-web-cf	prod-otk-pwrs-web-cf
Pricing plan	Free (월 1TB / 1천만 요청)	Standard
Price class	(Free 플랜에서 지정 불가)	PriceClass_200
Default root object	index.html	index.html
Alternate domain names	(선택)	powersales.otoki.com
Custom SSL certificate	(선택)	ACM 인증서 선택

Pricing plan 설명:

- **Free plan** (Dev): 월 1TB 데이터 송신 + 1천만 요청 + 100만 함수 호출까지 완전 무료. 한도 초과 시 throttling. **PriceClass 지정 자체가 금지되며** AWS가 엣지 라우팅을 자동 결정한다. dev 트래픽 수준에선 한도를 초과할 일이 없다.
- **Standard plan** (Prod): 일반 종량제. 한도 초과 시 차단되지 않으며 PriceClass로 엣지 범위 선택 가능. Prod는 한국 사용자 대상이므로 아시아 엣지를 포함하는 **PriceClass_200** 권장.

3.2 SPA 라우팅 설정 (필수)

콘솔: CloudFront > Distribution > Error pages > Create custom error response

HTTP Error Code	Response Page Path	HTTP Response Code
403	/index.html	200
404	/index.html	200

React SPA는 클라이언트 라우팅을 사용하므로, S3에 없는 경로 요청 시 `index.html` 을 반환해야 한다. 이 설정이 없으면 `/login`, `/dashboard` 등 직접 URL 접근 시 403/404 에러가 발생한다.

3.3 배포 후 캐시 무효화

빌드 후 CodeBuild에서 자동으로 invalidation을 수행한다. (`web/buildspec.yml` 에 포함)

```
aws cloudfront create-invalidation --distribution-id $DISTRIBUTION_ID --paths "/*"
```

전체 경로(/*) invalidation은 무료 1,000건/월 포함. 초과 시 건당 \$0.005.

3.4 배포 검증

1. CloudFront Distribution 상태가 **Deployed**로 변경될 때까지 대기 (약 5~15분)
2. CloudFront 도메인 (`https://d1234.cloudfront.net`) 접속하여 페이지 확인

3. SPA 라우팅 테스트: `https://d1234.cloudfront.net/login` 등 서브 경로 직접 접근 확인

4. S3 버킷 정책 설정

3장에서 CloudFront Distribution ID를 확보한 후 설정한다.

콘솔: S3 > `dev-otk-pwrs-web` > Permissions > Bucket policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontOAC",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::dev-otk-pwrs-web/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::<ACCOUNT_ID>:distribution/<DISTRIBUTION_ID>"
        }
      }
    }
  ]
}
```

<DISTRIBUTION_ID> 는 3장에서 CloudFront 생성 후 확인하여 입력한다. Prod도 동일 구조로 `prod-otk-pwrs-web` 버킷에 적용한다.

5. IAM — Web Role

3장에서 CloudFront Distribution ID를 확보한 후 진행한다.

5.1 Web CodeBuild Role (`dev-otk-pwrs-web-build-role`)

콘솔: IAM > Roles > Create role

1. Trusted entity: **AWS service** → **CodeBuild**
2. Role name: `dev-otk-pwrs-web-build-role`
3. 권한 정책 선택: 건너뛰기 (아무 정책도 선택하지 않음)
4. 역할 생성 후 > 권한 탭 > 권한 추가 > 인라인 정책 생성 > JSON
5. 인라인 정책 이름: `dev-otk-pwrs-web-build-policy`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "S3Deploy",
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ]
    }
  ]
}
```

```

    ],
    "Resource": [
        "arn:aws:s3:::dev-otk-pwrs-web",
        "arn:aws:s3:::dev-otk-pwrs-web/*"
    ]
},
{
    "Sid": "PipelineArtifacts",
    "Effect": "Allow",
    "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:GetBucketAcl",
        "s3:GetBucketLocation"
    ],
    "Resource": [
        "arn:aws:s3:::dev-otk-pwrs-pipeline-artifacts",
        "arn:aws:s3:::dev-otk-pwrs-pipeline-artifacts/*"
    ]
},
{
    "Sid": "CloudFrontInvalidation",
    "Effect": "Allow",
    "Action": "cloudfront:CreateInvalidation",
    "Resource": "arn:aws:cloudfront::<ACCOUNT_ID>:distribution/<DISTRIBUTION_ID>"
},
{
    "Sid": "CloudWatchLogs",
    "Effect": "Allow",
    "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
    ],
    "Resource": "arn:aws:logs:ap-northeast-2:<ACCOUNT_ID>:log-group:/aws/codebuild/*"
}
]
}

```

S3Deploy : *s3 sync --delete* 에 *PutObject*, *GetObject*, *DeleteObject*, *ListBucket* 모두 필요. *CloudFrontInvalidation* : 배포 후 캐시 무효화에만 사용하므로 *CreateInvalidation* 단일 액션으로 충분.

5.2 Web Pipeline Role (`dev-otk-pwrs-web-pipeline-role`)

콘솔: IAM > Roles > Create role

1. Trusted entity: **Custom trust policy**
2. 아래 신뢰 정책 JSON 입력:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "codepipeline.amazonaws.com"
      }
    }
  ]
}

```

```

    },
    "Action": "sts:AssumeRole"
  }
]
}

```

3. 권한 정책 선택: **건너뛰기** (아무 정책도 선택하지 않음)
4. Role name: `dev-otk-pwrs-web-pipeline-role`
5. 역할 생성 후 > 권한 탭 > **권한 추가** > **인라인 정책 생성** > JSON
6. 인라인 정책 이름: `dev-otk-pwrs-web-pipeline-policy`

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "GitHubConnection",
      "Effect": "Allow",
      "Action": [
        "codeconnections:UseConnection",
        "codeconnections:GetConnection",
        "codeconnections:GetConnectionToken",
        "codestar-connections:UseConnection",
        "codestar-connections:GetConnection",
        "codestar-connections:GetConnectionToken"
      ],
      "Resource": [
        "arn:aws:codeconnections:ap-northeast-2:<ACCOUNT_ID>:connection/*",
        "arn:aws:codestar-connections:ap-northeast-2:<ACCOUNT_ID>:connection/*"
      ]
    },
    {
      "Sid": "PipelineArtifacts",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:GetBucketAcl",
        "s3:GetBucketLocation"
      ],
      "Resource": [
        "arn:aws:s3:::dev-otk-pwrs-pipeline-artifacts",
        "arn:aws:s3:::dev-otk-pwrs-pipeline-artifacts/*"
      ]
    },
    {
      "Sid": "CodeBuildStart",
      "Effect": "Allow",
      "Action": ["codebuild:BatchGetBuilds", "codebuild:StartBuild"],
      "Resource": "arn:aws:codebuild:ap-northeast-2:<ACCOUNT_ID>:project/dev-otk-pwrs-web-build"
    },
    {
      "Sid": "SNSApproval",
      "Effect": "Allow",
      "Action": "sns:Publish",
      "Resource": "arn:aws:sns:ap-northeast-2:<ACCOUNT_ID>:dev-otk-pwrs-alerts"
    }
  ]
}

```

```
}  
]  
}
```

Backend Pipeline Role과 달리 AdministratorAccess-AWSElasticBeanstalk 관리형 정책이 불필요하다 — EB 배포가 없기 때문이다. Pipeline Artifact S3 버킷(`dev-otk-pwrs-pipeline-artifacts`)은 Backend(`infra-backend.md` 4.4)에서 생성한 버킷을 공유한다.

6. ACM 인증서 (us-east-1)

CloudFront는 글로벌 서비스이므로 **us-east-1** 리전의 ACM 인증서만 사용할 수 있다. Backend용 인증서(`infra-backend.md` 섹션 6)는 `ap-northeast-2`에 생성하지만, Web(CloudFront)용은 별도로 `us-east-1`에 생성해야 한다.

콘솔: 리전을 **us-east-1** (버지니아 북부)로 전환 > ACM (Certificate Manager) > Request certificate > Request a public certificate

Dev:

설정	값
Domain names	dev-powersales.otoki.com
Validation	DNS validation

Prod:

설정	값
Domain names	powersales.otoki.com
Validation	DNS validation

DNS validation을 선택하면 ACM이 CNAME 레코드를 제시한다. Route 53 호스팅 영역에 해당 CNAME을 등록하면 인증서가 발급된다.

7. Route 53 (DNS)

도메인이 있는 경우에만 해당한다. Route 53 호스팅 영역은 Backend에서 이미 생성되어 있다.

7.1 CloudFront에 도메인 연결

CloudFront 배포에 커스텀 도메인을 먼저 추가해야 Route 53에서 Alias 대상으로 선택할 수 있다.

콘솔: CloudFront > Distribution > General > Edit

설정	Dev	Prod
Alternate domain names	dev-powersales.otoki.com	powersales.otoki.com
Custom SSL certificate	ACM 인증서 선택	ACM 인증서 선택

ACM 인증서는 **us-east-1** 리전에 있어야 한다.

7.2 DNS 레코드

콘솔: Route 53 > Hosted zones > `otoki.com` > Create record

Record	Type	Alias Target
dev-powersales.otoki.com	A (Alias)	CloudFront 배포 도메인 (Dev)

powersales.otoki.com	A (Alias)	CloudFront 배포 도메인 (Prod)
----------------------	-----------	--------------------------

Alias Target으로 CloudFront 배포의 도메인 이름(`d1234.cloudfront.net`)을 선택한다.

8. CI/CD — Web (CodePipeline + CodeBuild)

GitHub push → CodePipeline → [Source] → [Build+Deploy: CodeBuild] → (Prod: Approval 후 실행)

Backend와 동일하게 CodePipeline으로 구성하여 승인 이력을 관리한다. CodeBuild는 npm 빌드 + S3 배포 + CloudFront 무효화를 담당하고, CodePipeline이 트리거와 승인을 관리한다. Pipeline Artifact S3 버킷(`dev-otk-pwrs-pipeline-artifacts`)은 Backend(`infra-backend.md` 4.4)에서 생성한 버킷을 공유한다.

8.1 Web CodeBuild 프로젝트 생성

콘솔: CodeBuild > Build projects > Create build project

CodeBuild는 CodePipeline에서 호출되므로 **Webhook**을 설정하지 않는다.

프로젝트 설정

항목	값
Project name	dev-otk-pwrs-web-build

소스

항목	값
Source provider	GitHub (CodeConnections 사용)
Connection	otk-pwrs-github (Backend에서 생성 완료)
Repository	codapt/otoki-code-build
Branch	dev
Buildspec	buildspec 파일 사용 — web/buildspec.yml

CodePipeline에서 호출 시 소스가 오버라이드되지만, "소스 없음"을 선택하면 buildspec 파일 참조가 불가하므로 GitHub 소스를 지정한다. Webhook은 설정하지 않는다 — CodePipeline이 트리거를 관리한다.

환경

항목	값
Environment image	Managed image
Operating system	Amazon Linux
Runtime	Standard
Image	aws/codebuild/amazonlinux-x86_64-standard:6.0
Compute	BUILD_GENERAL1_SMALL (3 GB RAM, 2 vCPU)
Privileged	체크 불필요 (Docker 미사용)
Service role	Existing service role → dev-otk-pwrs-web-build-role

Compute type 선택: Web은 Vite 빌드 + S3 sync + CloudFront invalidation만 수행하므로 Docker가 필요 없다. EC2 기반 최저 사양인 `BUILD_GENERAL1_SMALL` (3 GB / 2 vCPU)로 충분하다. 참고: CodeBuild Lambda compute(`BUILD_LAMBDA_2GB`)가 콜드 스타트 ~1초로 더 빠르지만 **ap-northeast-2 (서울) 리전에서는 미지원**(2026-04 기준)이다. 향후 서울 리전에 출시되면 전환 검토 가치가 있다.

환경변수

변수	값
S3_BUCKET	dev-otk-pwrs-web
DISTRIBUTION_ID	(CloudFront 배포 ID)

웹은 CloudFront 의 `/api/*` **path routing** 을 통해 ALB 로 프록시된다. 즉 프론트는 `fetch('/api/health')` 처럼 **상대 경로**로 호출하므로 브라우저 기준 `same-origin` 이 유지되며 **CORS 가 발생하지 않는다**. 이 때문에 `VITE_API_BASE_URL` 같은 absolute URL 주입이 필요 없다.

8.2 buildspec.yml (Web)

web/buildspec.yml 생성:

```
version: 0.2

phases:
  install:
    runtime-versions:
      nodejs: 22
  pre_build:
    commands:
      - cd web
      - npm ci
  build:
    commands:
      - npm run build
  post_build:
    commands:
      - aws s3 sync dist/ s3://$S3_BUCKET --delete
      - aws cloudfront create-invalidation --distribution-id $DISTRIBUTION_ID --paths "/*"
      - echo "Deployed to S3 and CloudFront invalidation created"
```

`npm ci` : `package-lock.json` 기반으로 정확한 의존성 설치 (CI 환경에 적합). `s3 sync --delete` : S3 버킷의 기존 파일 중 빌드 결과에 없는 파일을 삭제한다.

8.3 CodePipeline 생성 (Dev)

콘솔: CodePipeline > Pipelines > Create pipeline

Step 1 — Pipeline settings

설정	값
Pipeline name	dev-otk-pwrs-web-pipeline
Pipeline type	V2
Execution mode	Queued
Service role	Existing service role → dev-otk-pwrs-web-pipeline-role
Artifact store	Custom location → dev-otk-pwrs-pipeline-artifacts

Encryption key	Default AWS Managed Key (SSE-S3, KMS 사용 안 함)
----------------	---

Step 2 — Source stage

설정	값
Source provider	GitHub (GitHub 앱을 통해)
Connection	otk-pwrs-github (Backend에서 생성 완료)
Repository name	codapt/otoki-code-build
Default branch	dev
Output artifact format	CodePipeline default

Webhook 이벤트

설정	값
Webhook	체크 (푸시 및 풀 요청 이벤트)
이벤트 유형	푸시
필터 유형	브랜치
브랜치 패턴	dev
파일 경로	web/**

`dev` 브랜치에 푸시되고, `web/` 하위 파일이 변경된 경우에만 파이프라인이 트리거된다.

Step 3 — Build stage

설정	값
Build provider	AWS CodeBuild
Project name	dev-otk-pwrs-web-build
Input artifacts	SourceArtifact

Step 4 — Deploy stage

Web은 CodeBuild의 `post_build` 에서 S3 배포와 CloudFront 무효화를 처리하므로 별도 Deploy 스테이지가 불필요하다. Deploy stage 추가 화면에서 **Skip deploy stage**를 선택한다.

검증

1. CodePipeline 콘솔 > 파이프라인 선택 > **Release change** (수동 트리거)
2. Source → Build 각 스테이지 성공 여부 확인
3. Build 스테이지: CodeBuild 로그에서 `npm run build`, `s3 sync`, `cloudfront create-invalidation` 성공 확인
4. S3 버킷에 빌드 결과물(`index.html`, `assets/` 등) 확인
5. CloudFront URL로 접속하여 페이지 확인
6. SPA 라우팅 테스트: 서버 경로 직접 접근 확인

8.4 Prod CI/CD (CodePipeline)

Prod는 별도 CodePipeline + CodeBuild로 구성한다. Dev와 동일하되 아래 항목이 다르다.

항목	Dev	Prod
----	-----	------

파이프라인	dev-otk-pwrs-web-pipeline	prod-otk-pwrs-web-pipeline
소스 브랜치	dev	main
CodeBuild 프로젝트	dev-otk-pwrs-web-build	prod-otk-pwrs-web-build
Service role	dev-otk-pwrs-web-pipeline-role	prod-otk-pwrs-web-pipeline-role
환경변수	dev 리소스 참조	prod 리소스 참조

Prod 파이프라인은 수동 승인 스테이지를 추가한다:

Source → Approval → Build+Deploy

Approval 스테이지 설정

Source와 Build 사이에 **Manual approval** 액션을 추가:

설정	값
Action name	ManualApproval
Action provider	Manual approval
SNS topic ARN	arn:aws:sns:ap-northeast-2:<ACCOUNT_ID>:prod-otk-pwrs-alerts
Comments	Prod Web 배포를 승인하시겠습니까?

*main 브랜치 push 후 SNS 알림이 전송되고, 콘솔에서 승인해야 Build+Deploy 스테이지가 진행된다. Dev에서 이미 검증된 코드이므로 빌드 전 승인
으로 충분하다.*

9. CloudWatch (모니터링 & 로그)

9.1 로그 그룹

CodeBuild는 자동으로 CloudWatch Logs에 로그를 전송한다.

로그 그룹	소스
/aws/codebuild/dev-otk-pwrs-web-build	Dev Web CodeBuild
/aws/codebuild/prod-otk-pwrs-web-build	Prod Web CodeBuild

9.2 알람 설정

Stage	적용 여부	비고
Dev	미설정	비용 절감 및 알람 노이즈 방지를 위해 생략
Prod	설정	운영 장애 감지를 위해 필수

Prod 알람 구성

콘솔: CloudWatch > Alarms > Create alarm

알람	지표	조건
CloudFront 5XX 에러	5xxErrorRate (CloudFront)	> 5% (5분)

CloudFront 4XX 에러	4xxErrorRate (CloudFront)	> 30% (5분)
CodeBuild 실패	FailedBuilds (CodeBuild)	> 0 (1회)

알람 대상 SNS 토픽은 Backend(`infra-backend.md` 12.2)에서 생성한 `prod-otk-pwrs-alerts` 를 공유한다. Dev에는 SNS 토픽이 없다.

10. Dev vs Prod 차이점 요약

항목	Dev	Prod
S3 버킷	dev-otk-pwrs-web	prod-otk-pwrs-web
CloudFront	기본 도메인 사용 가능	커스텀 도메인 (powersales.otoki.com) 필수
CloudFront TTL	CachingOptimized (배포 시 invalidation)	CachingOptimized (배포 시 invalidation)
Pricing plan	Free (월 1TB / 1천만 요청)	Standard
Price class	지정 불가 (Free 플랜)	PriceClass_200
ACM 인증서	선택	필수 (us-east-1)
CI/CD	CodePipeline (dev push 자동)	CodePipeline (main push + 수동 승인)
SNS 알람 토픽	미설정	prod-otk-pwrs-alerts (Backend와 공유)
CloudWatch 알람	미설정	전체 구성 (9.2)
예상 월 비용	~\$1-5	~\$10-30

Frontend 비용은 Backend 대비 매우 낮다. S3 스토리지 + CloudFront 전송량 기준.

부록

자주 쓰는 AWS CLI 명령

```
# S3에 수동 배포
cd web && npm run build
aws s3 sync dist/ s3://dev-otk-pwrs-web --delete

# CloudFront 캐시 무효화
aws cloudfront create-invalidation \
  --distribution-id <DISTRIBUTION_ID> --paths "/*"

# CloudFront 배포 상태 확인
aws cloudfront get-distribution --id <DISTRIBUTION_ID> \
  --query "Distribution.Status"

# S3 버킷 파일 목록 확인
aws s3 ls s3://dev-otk-pwrs-web --recursive --human-readable

# CodeBuild 최근 빌드 확인
aws codebuild list-builds-for-project \
  --project-name dev-otk-pwrs-web-build --max-items 5
```

트러블슈팅

증상	원인	해결
CloudFront 403	OAC 미설정 또는 S3 버킷 정책 누락	OAC 생성 및 버킷 정책에 AllowCloudFrontOAC 추가
SPA 라우팅 안됨	CloudFront 커스텀 에러 응답 미설정	403/404 → /index.html (200) 설정 (3.2 참고)
배포 후 변경 안 보임	CloudFront 캐시	cloudfront create-invalidation --paths "/*" 실행
CodeBuild S3 sync 실패	IAM Role에 S3 권한 누락	dev-otk-pwrs-web-build-role의 S3Deploy 정책 확인
CodeBuild 빌드 실패	Node.js 버전 또는 의존성 문제	runtime-versions에서 nodejs 버전 확인, npm ci 로그 확인
HTTPS 안됨	ACM 인증서 미연결 또는 미검증	us-east-1에서 ACM 인증서 상태 확인, DNS validation 완료 확인

향후 고려사항

- **WAF**: Prod CloudFront 앞단에 WAF 적용 (DDoS 방어)
- **Lambda@Edge**: SEO 개선을 위한 서버사이드 렌더링 또는 리다이렉트 처리
- **Access Logging**: CloudFront 접근 로그를 S3에 저장하여 분석
- **Budget Alert**: CloudFront 트래픽 급증 시 비용 알람 설정